پروژه پایانی کارشناسی مهندسی برق- مخابرات

طراحی و پیاده‌سازی دیواره آتش هوشمند برنامه‌های تحت وب

یک Web Application Firewall با موتور تشخیص هیبرید
(Rule-Based + Machine Learning)



محدثه احمدی جیوان - شماره دانشجویی 9911522901

دانشگاه صداوسیما ـــ دانشکده فنی و مهندسی رسانه

WAF چیست؟

Web Application Firewall (WAF) یک لایه امنیتی برای محافظت از برنامه‌های وب است که درخواست‌های HTTP را پیش از رسیدن به برنامه بررسی می‌کند.

WAF

01 بررسی محتوای HTTP Request

02 فعالیت در Application Layer / Layer 7

03 بررسی بخش‌هایی مانند URL،Query و Body

04 قابلیت شناسایی الگوهای حملاتی مانند SQL Injection و XSS

Firewall سنتی

01 تمرکز عمده بر IP، Port و Protocol

02 فعالیت عمدتاً در لایه‌های شبکه و انتقال

03 تصمیم‌گیری بر اساس اطلاعات ارتباط شبکه





Payloadچیست؟

Payload بخش داده‌ای درخواست است که می‌تواند شامل ورودی کاربر و محتوای مورد بررسی Detector باشد.

در این پروژه، Payload از بخش‌های مختلف Request استخراج و برای تشخیص به موتور WAF داده می‌شود.

هدف و محدوده پروژه

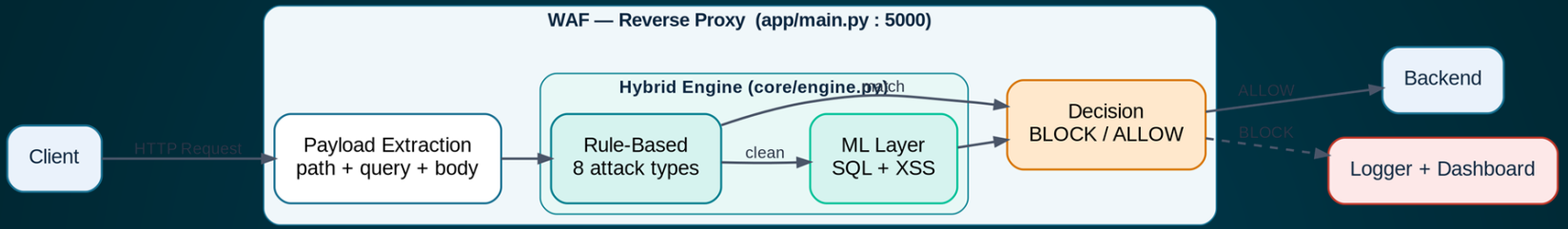
هدف پروژه، طراحی و پیاده‌سازی یک **WAF** کاربردی و آزمایشگاهی است که بتواند درخواست‌های **HTTP** را قبل از رسیدن به **Backend** دریافت، تحلیل و درباره آن‌ها تصمیم‌گیری کند.

- استفاده از Machine Learning برای تشخیص SQL Injection و XSS
- تشخیص ۸ سناریوی حمله در لایه Rule-Based
- قرارگیری WAF به‌صورت Reverse Proxy جلوی Backend
- ارزیابی عملکرد با مجموعه تست مستقل
- ثبت رویدادهای امنیتی و نمایش آن‌ها در Dashboard
- اجرای اجزای اصلی در Docker

خارج از محدوده فعلی

پیاده‌سازی کامل **HTTPS/TLS**، استقرار **Production** در مقیاس واقعی و تضمین تشخیص تمام حملات کاملاً ناشناخته در محدوده این **Prototype** قرار ندارند.

معماری کلی سیستم



WAF در این پروژه به‌صورت Reverse Proxy بین Client و Backend قرار گرفته است؛ بنابراین Client مستقیماً با Backend ارتباط برقرار نمی‌کند.

هر درخواست ابتدا توسط WAF دریافت شده و پس از استخراج Payload وارد فرآیند تشخیص می‌شود.

نتیجه تصمیم

BLOCK : درخواست به Backend ارسال نمی‌شود و رویداد در Log ثبت می‌شود.

ALLOW : درخواست به Backend Forward می‌شود و پاسخ Backend به Client بازگردانده می‌شود.

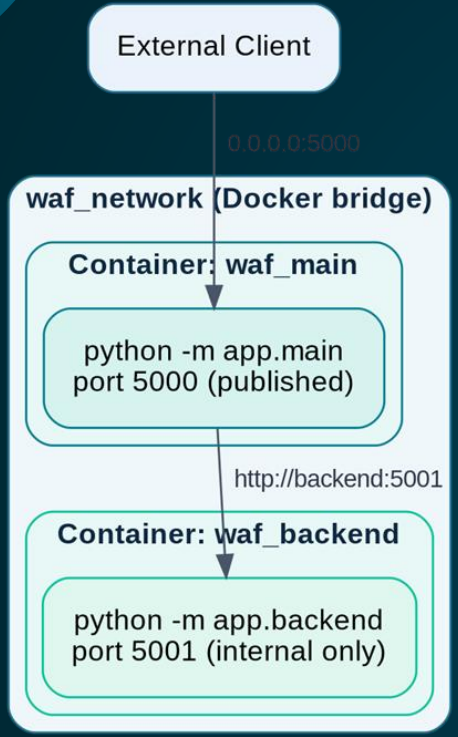
Reverse Proxy چیست؟

Reverse Proxy سرویسی است که از طرف Server مقصد درخواست‌های Client را دریافت می‌کند و می‌تواند پیش از Forward کردن درخواست، آن را بررسی و کنترل کند.

ساختار پروژه

app/	سرورهای Flask: main.py (WAF)، backend.py، dashboard_api.py	دریافت Request و سرویس‌های Flask
core/	engine.py (تصمیم‌گیری Hybrid)، blocker.py، logger.py	موتور تصمیم‌گیری Hybrid، Blocking و Logging
rules/	۸ Detector قاعده‌محور (Regex + شمارش نشانه)	۸ Detector های Rule-Based برای ۸ سناریوی حمله
ml/	train.py، SQL/XSS، مدل‌های features.py	Feature Engineering، آموزش و مدل‌های SQL/XSS
data/datasets/	تولید دیتاست مصنوعی SQL و XSS	داده‌های آموزشی مصنوعی
evaluation/	مجموعه تست مستقل ۱۱۹ نمونه‌ای + metrics.py	مجموعه تست مستقل و محاسبه Metrics
dashboard/	رابط HTML مانیتورینگ زنده	رابط مانیتورینگ و نمایش Logهٔ

بسته بندی با Docker



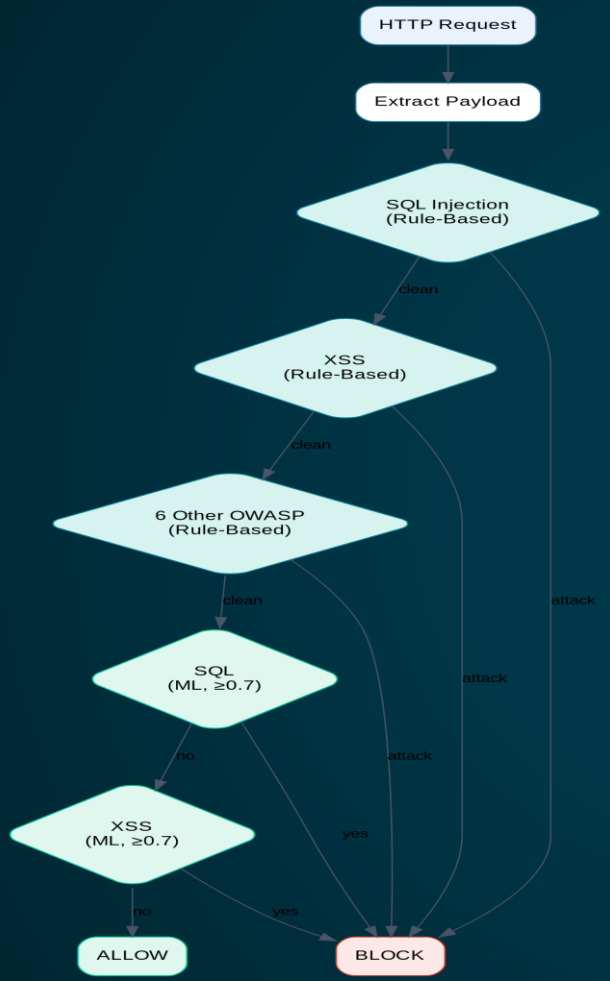
برای جداسازی WAF از برنامه مقصد، اجزای اصلی سیستم در دو Container اجرا می‌شوند.

waf_main
نقطه ورود Request
پورت ۵۰۰۰
در دسترس Client

waf_backend
برنامه وب مقصد
پورت ۵۰۰۱
دسترسی داخلی از طریق شبکه Docker

در این معماری، Backend مستقیماً در معرض Client قرار نمی‌گیرد و Request ها ابتدا از WAF عبور می‌کنند.Container محیط اجرایی ایزوله‌ای است که برنامه و وابستگی‌های آن را در یک محیط قابل‌تکرار اجرا می‌کند.

چرخه کامل Request



چرخه کامل یک درخواست از Payload تا تصمیم :

در این پروژه، Hybrid به معنی یک Sequential Detection Pipeline است؛ یعنی Detector ها به‌صورت متوالی اجرا می‌شوند و با یکدیگر رأی‌گیری یا Ensemble نمی‌کنند.

ترتیب پردازش

HTTP Request↓
Payload Extraction↓
SQL Rule↓
XSS Rule↓
شش Rule دیگر ↓
SQL ML↓
XSS ML↓
ALLOW / BLOCK

به‌محض اینکه یک Detector حمله را تشخیص دهد، بررسی متوقف شده و Request مسدود می‌شود.در صورت عدم تشخیص حمله، Request در نهایت به Backend Forward می‌شود.

Payload از کجا ساخته می‌شود؟

برای اینکه Detector ها یک ورودی واحد داشته باشند، بخش‌های مختلف HTTP Request استخراج و با یکدیگر ترکیب می‌شوند:

- Path: مسیر درخواست
- Query String: رشته پارامترها بعد از ?
- Query Parameters: پارامترهای استخراج‌شده از Query
- Body: محتوای بدنه Request

پس از Decode شدن این بخش‌ها، یک Payload واحد ساخته می‌شود:

Path + Query String + Query Parameters + Body

این Payload به Detector های Rule-Based و در صورت نیاز به مدل‌های ML ارسال می‌شود.

```
decoded_path = unquote(path)
query_string =
unquote(request.query_string.decode())
query_params = " ".join(f"{k}={v}" for k, v
in request.args.items())
body = request.get_data(as_text=True)

payload = " ".join([decoded_path,
query_string, query_params, body])
```

هشت سناریوی Rule-Based

نوع حمله	فایل مسئول	روش تشخیص
SQL Injection	rules/sql_advanced.py	شمارش کلمات کلیدی + Regex
XSS	rules/xss_advanced.py	چندمرحله‌ای Regex + Decode
Command Injection	owasp_other_advanced.py	Regex
LDAP Injection	owasp_other_advanced.py	شمارش کاراکتر خاص
XXE	owasp_other_advanced.py	شمارش کلمات کلیدی XML
Path Traversal	owasp_other_advanced.py	Regex
SSRF	owasp_other_advanced.py	شمارش رشته‌های آدرس داخلی
Open Redirect	owasp_other_advanced.py	شمارش شاخص URL

چگونه SQLInjectionDetector امتیاز می‌دهد؟

Detector ابتدا Payload را با مجموعه‌ای از Regex ها بررسی می‌کند.هر تطابق:
confidence +۰.۳
سپس Keyword های SQL نیز بررسی می‌شوند و در صورت وجود حداقل دو Keyword ، امتیاز بیشتری اضافه می‌شود.
مثال:
OR 1=1 --

در این Payload:
Boolean-based → +0.3
Comment → +0.3

بنابراین:
confidence = 0.6
و چون آستانه Rule برابر ۰.۵ است:
BLOCK → ۰.۵ ≤ ۰.۶

این روش تلاش می‌کند تصمیم را بر اساس ترکیب چند نشانه بگیرد، اما همین Threshold می‌تواند باعث از دست رفتن برخی Payload های ساده شود.

```
for pattern in self.patterns:
if re.search(pattern, upper_input):
        confidence += 0.3


keyword_count = sum(1 for kw in
self.sql_keywords if kw in upper_input)
if keyword_count >= 2:
    confidence += 0.2 * keyword_count
```

چرا ML در کنار Rule-Based؟

Rule-Based بر اساس الگوهایی عمل می‌کند که از قبل تعریف شده‌اند.
بنابراین تغییر شکل یک Payload ممکن است باعث شود یک Regex مشخص دیگر Match نشود.

در این پروژه، ML به‌عنوان لایه تشخیص مکمل اضافه شده است.
مدل با استفاده از ویژگی‌هایی مانند:

- طول Payload
- تنوع کاراکترها
- تعداد کاراکترهای خاص
- Keyword های مرتبط با SQL و XSS

احتمال حمله بودن Payload را برآورد می‌کند.



نکته

ML جایگزین Rule-Based نیست.
در نسخه فعلی، MLفقط برای:SQL Injection + XSS استفاده شده است.
مدل Logistic Regression این پروژه از Deep Learning یا تحلیل معنایی متن استفاده نمی‌کند.

Feature Engineering ۲۰ ویژگی واقعی

در این مرحله Payload خام به مجموعه‌ای از ویژگی‌های قابل استفاده برای مدل تبدیل می‌شود.

۱۱

پرچم کلمه کلیدی SQL
UNION، SELECT،
DROP...

۶

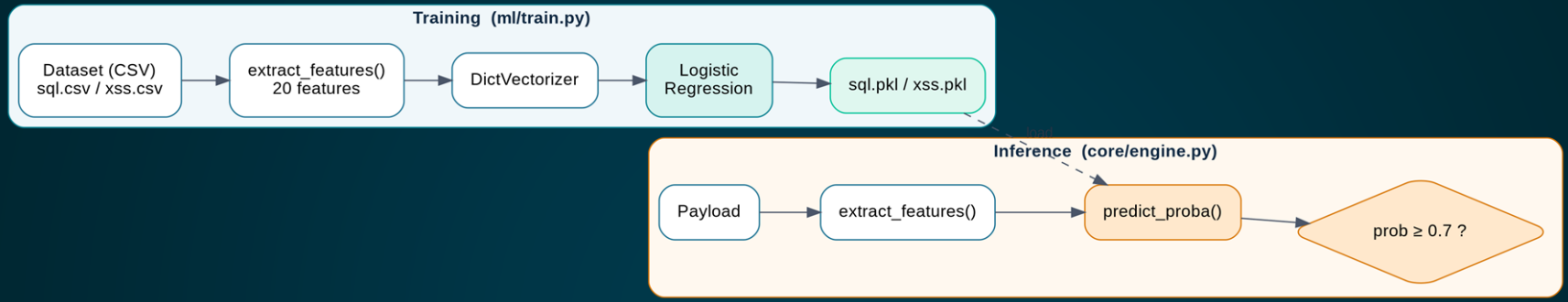
پرچم کلمه کلیدی XSS
<script>، onerror...

۳

ویژگی عمومی
length، entropy،
special_chars

```
def extract_features(payload):
    features = {}
    features["length"] = len(payload)
    features["entropy"] = entropy(payload)
    for kw in SQL_KEYWORDS:  features[f"sql_{kw}"] = 1 if kw in payload else 0
    for kw in XSS_KEYWORDS:  features[f"xss_{kw}"] = 1 if kw in payload else 0
```

Pipeline آموزش و Inference



Training--آموزش مدل

Dataset → Feature Extraction → DictVectorizer → Logistic Regression → Saved Model

در ml/train.py ویژگی‌های Dataset استخراج شده و مدل‌های SQL و XSS آموزش داده و ذخیره می‌شوند.

Inference استفاده از مدل

New Request → Feature Extraction → Trained Model → predict_proba() → Decision

در زمان اجرای WAF، مدل ذخیره‌شده برای Payload جدید احتمال تعلق آن به کلاس حمله را محاسبه می‌کند.

Training: یادگیری پارامترهای مدل از داده‌های آموزشی

Inference: استفاده از مدل آموزش‌دیده برای پیش‌بینی روی داده جدید

آستانه تصمیم ML



مدل با استفاده از predict_proba() احتمال تعلق Payload به کلاس حمله را محاسبه می‌کند.

در پیاده‌سازی فعلی:

Probability ≥ 0.7 → BLOCK

Probability < 0.7 → عدم Block توسط ML

عدد ۰.۷ یک انتخاب طراحی در این Prototype است و با یک فرآیند مستقل برای تعیین Threshold بهینه آماری انتخاب نشده است.
بنابراین این مقدار به‌عنوان بهترین Threshold معرفی نمی‌شود.

Dataset آموزشی

مدل‌های SQL و XSS با Datasetهای مصنوعی تولیدشده برای پروژه آموزش داده شده‌اند.

41	32 / 21	6000+6000
الگوی منحصربه‌فرد ترافیک معمولی	الگوی منحصربه‌فرد حمله SQL / XSS	سطر (sql.csv + xss.csv)

محدودیت Dataset

داده‌ها Synthetic هستند و از ترافیک واقعی Production جمع‌آوری نشده‌اند.در طول توسعه، پس از مشاهده رفتار سیستم، نمونه‌های معمولی متنوع‌تری اضافه و مدل‌ها مجدداً آموزش داده شدند.

فرآیند توسعه:

Test → → افزودن نمونه → مشاهده ضعف Retraining → Test مجدد

سناریوی تست زنده



چهار درخواست واقعی به سرور در حال اجرا ارسال می‌شود؛ نتیجه هرکدام بلافاصله در ترمینال و داشبورد دیده می‌شود.

تست ۱ و ۲ :حملات Rule-Based

```
GET localhost:5000/?id=1' OR 1=1 --
```

Boolean-based + Comment الگوی--SQLInjectionDetector

BLOCK

```
GET localhost:5000/?q=<script>alert(1)</script>
```

شناخته‌شده<script> تگ --XSSDetector

BLOCK

تست ۳ و ۴ :تشخیص ML و یک درخواست سالم

```
GET localhost:5000/?u=admin'--
```

SQL Injection (ML-Based)

BLOCK

```
GET localhost:5000/?search=book
```

نه Rule و نه ML چیزی مشکوک نمی‌یابند ← درخواست به Backend می‌رسد

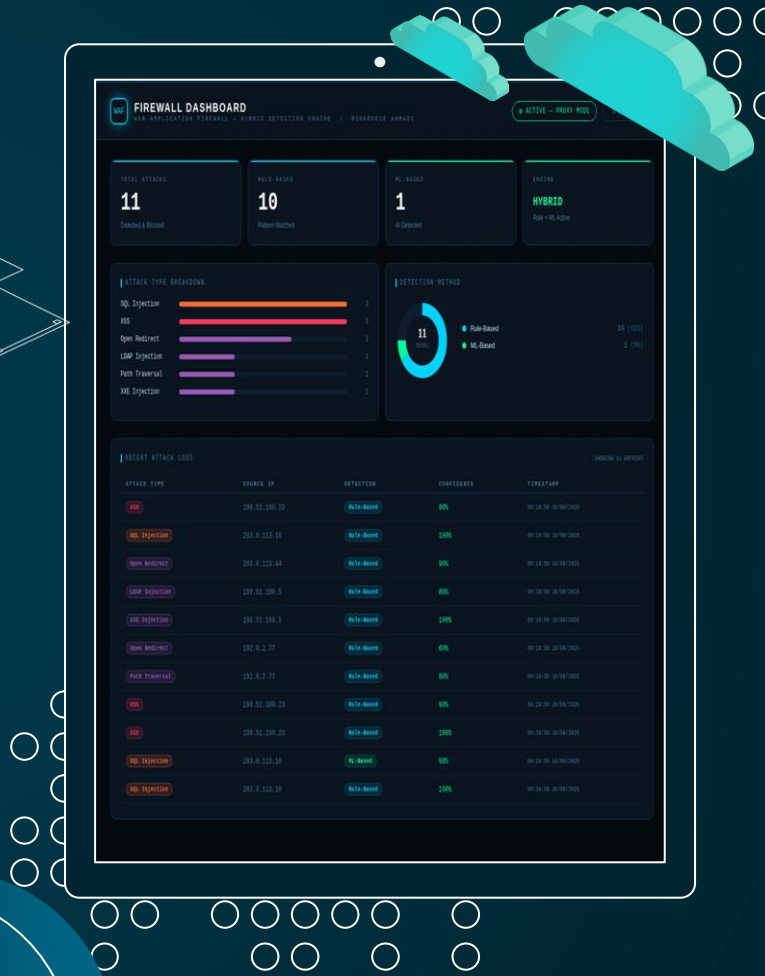
ALLOW

Dashboardچگونه داده دریافت می‌کند؟



رویدادهای ثبت‌شده توسط WAF در waf.log ذخیره می‌شوند
.سپس API های موجود در dashboard_api.py اطلاعات Log را در اختیار Dashboard قرار می‌دهند

.بنابراین اطلاعات نمایش‌داده‌شده در Dashboard حاصل Logging واقعی سیستم است.



روش ارزیابی: مجموعه تست مستقل

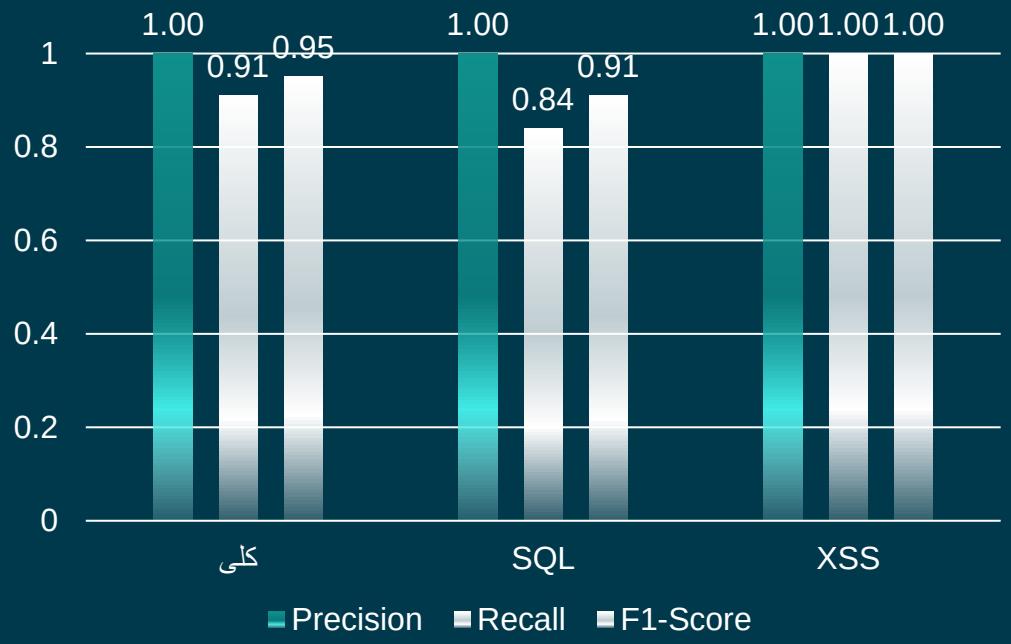
برای اینکه عملکرد WAF فقط با چند Demo نشان داده نشود، یک مجموعه تست مستقل از Dataset آموزشی ایجاد و اجرا شد.

0	52 + 67	119
همپوشانی با Dataset آموزشی	حمله+ معمولی (38 SQL, 29 XSS)	نمونه مستقل جدا از Dataset آموزشی

جداسازی داده‌ها

مجموعه evaluation/test_dataset.csv با Dataset های آموزشی sql.csv و xss.csv همپوشانی ندارد.
تمام ۱۱۹ نمونه واقعاً از:
HybridEngine.inspect()
عبور داده شدند و خروجی سیستم با Label واقعی مقایسه شد.
Metrics نیز توسط calculate_metrics در evaluation/metrics.py محاسبه شدند.

نتایج واقعی ارزیابی



6 FN

از ۶۷ حمله — همه از نوع SQL

0 FP

روی ۵۲ درخواست معمولی
False Positive Rate = 0.00

از مجموع ۱۱۹ نمونه مستقل:

ترافیک معمولی

۵۲ / ۵۲ صحیحاً **ALLOW** شدند
False Positive = 0
False Positive Rate = 0.00

حملات

۶۱ / ۶۷ صحیحاً **BLOCK** شدند
۶ مورد **False Negative**
هر ۶ مورد False Negative مربوط به **SQL Injection** بودند.

Metrics کلی

Precision = 1.00
Recall = 0.91
F1-Score = 0.95

تحلیل موارد ازدست‌رفته (Failure Analysis)

admin' #	یک نشانه (Comment #) به‌تنهایی به آستانه ۰.۵ نمی‌رسد
1 AND 1=1	مشابه بالا؛ فقط یک الگوی Boolean-based فعال می‌شود
SeLeCt * FrOm users	بدون کوتیشن/UNION/کامنت — از نظر نحوی شبیه یک کوئری عادی است
1/**/OR/**/1=1	تکنیک شناخته‌شده دور زدن WAF: کامنت خالی به‌جای فاصله واقعی، الگوی Regex را رد می‌کند

Precision: از میان مواردی که سیستم Attack تشخیص داده، چند مورد واقعاً Attack بوده‌اند؟
Recall: از میان تمام Attack های موجود، چند مورد توسط سیستم شناسایی شده‌اند؟
F1-Score: میانگین هارمونیک Precision و Recall است و عملکرد این دو معیار را به‌صورت همزمان خلاصه می‌کند

برداشت از نتایج این تست در مجموعه ارزیابی:
False Positive مشاهده نشد؛ در مقابل، تعدادی از SQL Injectionها شناسایی نشدند.

محدودیت‌های شناخته‌شده و مسیر توسعه

محدودیت‌های فعلی

- Dataset مصنوعی و با تنوع محدود است
- برخی SQL Injection و Evasion ها شناسایی نمی‌شوند
- HTTPS/TLS کامل پیاده‌سازی نشده
- verify=False در Forward فعلی وجود دارد
- سرویس‌ها با debug=True اجرا می‌شوند
- Query String در Forward فعلی حفظ نمی‌شود
- مجموعه Evaluation محدود است

مسیر توسعه آینده

- استفاده از Dataset متنوع‌تر و نزدیک‌تر به ترافیک واقعی
- بهبود Regex و Normalization
- اضافه‌کردن TLS و Hardening
- حذف آن پیش از Production
- غیرفعال‌کردن Debug در Production
- اصلاح Forward و حفظ Query String
- افزایش حجم و تنوع مجموعه تست مستقل

وضعیت فعلی

این سیستم یک Functional Prototype آزمایشگاهی است و برای استقرار Production، اصلاح موارد امنیتی و عملیاتی فوق ضروری است.

جمع‌بندی

در این پروژه یک Web Application Firewall کاربردی و آزمایشگاهی طراحی و پیاده‌سازی شد.

آنچه پیاده‌سازی شد

- معماری Reverse Proxy بین Client و Backend
- Pipeline ترکیبی Sequential Rule-Based + ML
- تشخیص ۸ سناریوی حمله در لایه Rule-Based
- استفاده از ML برای SQL Injection و XSS
- دو Container مجزا با Docker
- Logging و Dashboard برای Monitoring
- مجموعه تست مستقل شامل ۱۱۹ نمونه

نتیجه Evaluation

Precision = 1.00
Recall = 0.91
F1-Score = 0.95

همچنین Failureهای مشاهده‌شده، به‌ویژه در SQL Injection، بررسی و برای هرکدام مسیر توسعه مشخص شده است

وضعیت نهایی

Functional Prototypeآزمایشگاهی — نه محصول آماده Production